

INFORME TECNICO DE ARQUITECTURA

Fitness Microservices System

Este documento presenta una guía completa del sistema de microservicios ampliado para la plataforma de Fitness. Explica la arquitectura final de 6 microservicios de negocio, los mecanismos internos, como levantar el entorno y como realizar las pruebas de funcionamiento de extremo a extremo.

1. Arquitectura y Componentes del Sistema

El sistema esta diseñado bajo una arquitectura de microservicios descentralizada apoyada en el ecosistema de Spring Cloud y asegurada con OAuth2 (Keycloak). Cuenta con 6 microservicios de negocio y 4 configuraciones/soporte.

Diagrama de Arquitectura de Microservicios

(Ver diagrama de flujo visual interactivo en la version Markdown: C:\Users\mauri\geminil\antigravity\brain\92ca0d4d-2fec-42e4-a52a-382ae7025c50\project_report.md)

Puertos y Modulos del Proyecto

Modulo	Puerto	Descripcion	Base de Datos / Almacenamiento
`eureka-server`	`8761`	Servidor de Descubrimiento de Netflix Eureka.	En memoria
`config-server`	`8888`	Servidor de Configuracion Centralizada (Perfil `native`).	Sistema de archivos
`api-gateway`	`8080`	Puerta de entrada unificada. Enruta y valida los tokens de s	En memoria
`usuario-service`	`8081`	Gestiona unicamente usuarios.	MySQL (`usuario_db`)
`membresia-service`	`8084`	Gestiona membresias independientes.	MySQL (`membresia_db`)
`rutina-service`	`8082`	Gestiona rutinas independientes de entrenamiento.	MySQL (`rutina_db`)
`ejercicio-rutina-service`	`8085`	Gestiona ejercicios individuales asignados a rutinas.	MySQL (`ejercicio_rutina_db`)
`progreso-service`	`8083`	Registra el historico de peso y avance fisico.	MySQL (`progreso_db`)
`nutricion-service`	`8086`	Gestiona los planes de alimentacion diaria.	MySQL (`nutricion_db`)
`keycloak`	`8180`	Servidor de identidad y acceso OAuth2.	H2 integrada
`mysql`	`3306`	Servidor de bases de datos relacionales comun.	Persistencia en Docker volume

2. Mecanismos de Comunicacion Inter-servicio y Propagacion

A. Interceptor Feign para Propagacion de Tokens

Cuando un microservicio (como membresia-service o ejercicio-rutina-service) necesita llamar a otro (como usuario-service o rutina-service) a traves de Feign, debe propagar el token de seguridad Bearer. Esto se logra mediante el FeignClientInterceptor configurado en cada servicio que actua como cliente:

```
@Component
public class FeignClientInterceptor implements RequestInterceptor {
```

```

@Override
public void apply(RequestTemplate template) {
    ServletRequestAttributes attributes = (ServletRequestAttributes) RequestContextHolder.getRequestAttributes();
    if (attributes != null) {
        String authHeader = attributes.getRequest().getHeader("Authorization");
        if (authHeader != null && authHeader.startsWith("Bearer ")) {
            template.header("Authorization", authHeader);
        }
    }
}
}
}

```

B. Validaciones Cruzadas Implementadas

- membresia-service -> usuario-service: Valida que el idUsuario exista en usuario-service antes de registrar la membresia.
- rutina-service -> usuario-service: Valida que el idUsuario exista antes de registrar la rutina.
- ejercicio-rutina-service -> rutina-service: Valida que la rutina (idRutina) exista en rutina-service antes de asignarle un ejercicio.
- progreso-service -> usuario-service: Valida que el idUsuario exista antes de registrar el progreso.
- nutricion-service -> usuario-service: Valida que el idUsuario exista antes de registrar el plan alimentario.

3. Guia de Ejecucion del Proyecto

Instrucciones de Arranque

- Abre una terminal de Windows en la raiz del proyecto:

powershell

cd "C:\Users\mauri\Downloads\fitness-microservices\fitness-microservices"

- Compila el proyecto:

powershell

mvn clean package -DskipTests

- Levanta el entorno con Docker:

powershell

docker-compose up --build -d

4. Guia Completa de Pruebas de APIs (Flujo Logico)

Paso 1: Obtener el Token de Keycloak

- Metodo: POST
- URL: http://localhost:8180/realms/fitness-realm/protocol/openid-connect/token
- Body (x-www-form-urlencoded):
 - * client_id : fitness-client
 - * client_secret : fitness-client-secret
 - * grant_type : password

- * username : admin
- * password : admin123
- *Copia el access_token obtenido.*

Paso 2: Crear un Usuario (usuario-service)

- Metodo: POST
- URL: http://localhost:8080/api/usuarios
- Headers: Authorization: Bearer <TOKEN>
- Body (JSON):

```
json
{
  "nombre": "Carlos Mendoza",
  "email": "carlos.m@mail.com",
  "telefono": "987654321",
  "fechaRegistro": "2026-07-27"
}
```

- *Asumiremos que el ID retornado es 1.*

Paso 3: Asignar una Membresia (membresia-service)

- Metodo: POST
- URL: http://localhost:8080/api/membresias
- Headers: Authorization: Bearer <TOKEN>
- Body (JSON):

```
json
{
  "idUsuario": 1,
  "plan": "Premium Anual",
  "costo": 350.00,
  "inicio": "2026-07-27",
  "fin": "2027-07-27",
  "estado": "Activo"
}
```

Paso 4: Crear una Rutina (rutina-service)

- Metodo: POST
- URL: http://localhost:8080/api/rutinas
- Headers: Authorization: Bearer <TOKEN>
- Body (JSON):

```
json
{
  "nombreRutina": "Rutina Hipertrofia Fuerza",
  "idUsuario": 1
}
```

- *Asumiremos que el ID retornado es 1.*

Paso 5: Asignar Ejercicio a la Rutina (ejercicio-rutina-service)

Este paso valida mediante Feign contra rutina-service que la rutina con ID 1 exista.

- Metodo: POST
- URL: `http://localhost:8080/api/ejercicios-rutina`
- Headers: Authorization: Bearer <TOKEN>
- Body (JSON):

```
json
{
  "nombre": "Press de Banca Plano",
  "series": 4,
  "idRutina": 1
}
```

Paso 6: Registrar Progreso (progreso-service)

- Metodo: POST
- URL: `http://localhost:8080/api/progresos`
- Headers: Authorization: Bearer <TOKEN>
- Body (JSON):

```
json
{
  "idUsuario": 1,
  "fechaRegistro": "2026-07-27",
  "peso": 78.50,
  "porcentajeGrasa": 15.4,
  "porcentajeMusculo": 42.1
}
```

Paso 7: Crear Plan de Nutricion (nutricion-service)

- Metodo: POST
- URL: `http://localhost:8080/api/nutriciones`
- Headers: Authorization: Bearer <TOKEN>
- Body (JSON):

```
json
{
  "planDetalle": "Plan Proteico para Hipertrofia",
  "caloriasDiarias": 2800,
  "objetivo": "Aumento de masa muscular",
  "idUsuario": 1
}
```